

Finding (opting for) a strategy and the efficiency factor

While designing strategies it is important to consider their viability, effectiveness and efficiency. To comprehend the idea completely, consider a basic problem in quantum mechanics.

Schrödinger equation for the time-dependent wave function can be written as:

$$i\hbar \frac{\partial}{\partial t} |\Phi(t)\rangle = H |\Phi(t)\rangle$$

We can also write an expression for the thermal expectation value of an observable X as:

$$\langle X \rangle = \frac{\text{Tr} e^{-\beta H} X}{\text{Tr} e^{-\beta H}}$$

You can see that the above equation is modelled by a Hamiltonian H. Classically, it is quite easy to come out with a computational method to solve such equations (say by using Monte Carlo methods). But here the problem is that the objects (say operators or matrices) in QM do not necessarily commute.

Still, we can go for models defined by:

$$\langle X \rangle = \sum_{i=0}^{M-1} e^{-\beta E_i} \langle \Phi_i | X | \Phi_i \rangle / \sum_{i=0}^{M-1} e^{-\beta E_i}$$

A lattice of L sites filled with L/2 electrons with up spin, and L/2 electrons with down spin, is a physical model that easily fits into this. (Please Google the term 'Hubbard model' for more information about a better model.) But to find out what is really required to carry out these few steps, we need an order of magnitude for M. And by using approximation methods (like the Sterlings method) we can see that:

$$M \approx \frac{2^{2L+2}}{2\pi L}$$

This means that the quantity M increases exponentially with 2L (approximately). And if we allocate 8 bytes per floating point number, the amount of memory we need to store a single eigenvector will turn out to be:

$$\text{MEMORY} \approx \frac{2^{2L+2.5}}{2\pi L} \text{Gb}$$

So if I put L = 64, the memory required will be 1028 GB! This means that I need 1028 GB to study a quantum system of just 64 particles on 64 sites. If I submit a proposal with such high values, I am sure that no funding agency will accept this.

The only way I can do the computational task is to go for an algorithmic strategy that will reduce the amount of memory needed, at the expense of more CPU time. This is further considered in relation to 'clouds' and their effectiveness.

line: 330 90 1000, 330 -90 1000

Here the X and Y values (which you can identify from the codes directly) are in degrees around the sphere. And Z (or some axis reference) is the sphere's radius. As you can see, we have used different colours for east-west lines and north-south lines.

Now we will create our flat grid file from this, using the following shell code:

```
#!/bin/sh
/path_to_holodraw/drawwrt.pl < /location_of_input_file/sphere.draw >
flatgrid.wrt
```

But when we draw the sphere, we have to slice our long lines into small ones, so that our sphere will have a 'smooth' curve. We can do that by using the 'drawchop' and 'drawball' library files:

```
#!/bin/sh
/path_to_holodraw/drawchop.pl x=15+15 y=15+15 < /location_of_input_
file/sphere.draw |
/path_to_holodraw/drawball.pl |
/path_to_holodraw/drawwrt.pl > ballgrid.wrt
```

We can create the VRML (Virtual Reality Modelling Language) using the 'drawwrt' file:

```
#VRML V2.0 utf8
# draw a line around the sphere's equator
Shape {
  appearance Appearance {
    material Material {
      emissiveColor 0 1 0
      transparency 0
    }
  }
  geometry IndexedLineSet {
    coord Coordinate {
      point [
        0 0 1000,
        500 0 866.025403784439,
        866.025403784439 0 500,
        1000 0 6.12303176911189e-14,
        866.025403784439 -0 -500,
        500 0 -866.025403784439,
        1.22460635382238e-13 0 -1000,
        -500 0 -866.025403784439,
        -866.025403784438 0 -500,
        -1000 0 -1.83690953073357e-13,
        -866.025403784439 0 500,
        -500 0 866.025403784438,
        -2.44921270764475e-13 0 1000
      ]
    }
    coordIndex [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 ]
  }
}
```